

Proyecto 02: Gestión de un Hospital [Servicio médico].

Este documento contiene las especificaciones de funcionalidad y requerimientos para el segundo proyecto de programación de la asignatura **Estructura de Datos y de la Información** en el curso 2022/23.

1. Introducción.

Nuestro objetivo será desarrollar una aplicación que realice la gestión básica de personal y atención de un servicio especializado en un hospital. La aplicación tendrá una funcionalidad muy limitada, pero podrá ser extendida posteriormente. Por tanto, debe cumplir con las siguientes características generales:

- **Eficiencia:** debido a la gran cantidad de datos y al esperado crecimiento de los mismos en el futuro, debemos encontrar la forma de almacenar y procesar los datos de la forma más eficiente posible. Además, deberemos realizar un análisis de la complejidad de nuestra aplicación para cumplir este requisito.
- **Extensibilidad:** el diseño de la aplicación en cuanto a entidades y estructuras de datos debe ser extensible, para facilitar el desarrollo de futuras versiones de la aplicación y ampliar su funcionalidad.
- **Modularidad:** la organización de las entidades que forman nuestra aplicación debe ser modular, de forma que las modificaciones y ampliaciones de la misma se puedan realizar de la forma más sencilla posible, afectando a la menor cantidad de código.

Para cumplir con los requisitos anteriores se seguirá una metodología de desarrollo basada en el paradigma de *Programación Orientada a Objetos* y se utilizará el lenguaje de programación **C++**.

Este documento se organiza de la siguiente forma. En la **sección 2**, se especifica el código a partir del cual se puede comenzar la implementación del proyecto. En la **sección 3**, se enumeran los algoritmos básicos que debemos implementar y, en la **sección 4**, se discuten los mecanismos de análisis y tests que debemos aplicar sobre nuestra aplicación para evaluarla, y asegurar que cumple los requisitos y lo hace de forma eficiente. Finalmente, la **sección 5**, establece los criterios de evaluación de la calidad del desarrollo y su calificación, así como las fechas de entrega.

2. Creación del código base para el proyecto.

El proyecto contendrá las clases implementadas durante las sesiones de laboratorio (algunas proporcionadas por el profesorado y disponibles en el aula virtual):

1. **Paciente, Médico, Informe, FechaYHora, Servicio y Hospital.**
2. La clase **Paciente**, para esta entrega, debe tener un nuevo atributo para almacenar todos los informes que se vayan generando, ordenados desde el más reciente al más antiguo. Para este atributo se deben implementar los métodos necesarios para su gestión (añadir un nuevo informe, mostrar todos los informes etc.)
3. **Informe:** esta clase debe tener los atributos necesarios para el texto del informe, la fecha y hora del informe y los datos del médico que lo realizó.
4. **Médico:** esta clase debe tener los atributos necesarios para guardar su nombre, apellidos y especialidad.

5. `ListaPacientes`, `ListaMedicos`, `PilaInformes` y `ColaPacientes` son las clases contenedoras de la información.
6. **Servicio**: Esta clase representa el comportamiento de un servicio médico de un hospital, compuesto por el médico responsable del servicio y las colas de prioridad de pacientes, llegada y atención de pacientes. Tendrá como **atributos** el nombre de la **especialidad**, el **médico** responsable del servicio y la **información de los pacientes** en espera de ser atendidos en orden a su prioridad y llegada. El servicio será el encargado de **procesar** (atender) sus colas de prioridad de pacientes. (*Ver documento sesión 06 prácticas*).

En el momento de cargar los datos de los pacientes desde el fichero de texto, a cada uno de ellos se les debe asignar una prioridad de forma aleatoria. De este modo, cada paciente, según su prioridad, se almacenará en la cola correspondiente del servicio, en espera a ser atendido.

7. La clase **Hospital**. Es la clase principal del proyecto. Será la encargada de **crear los objetos que se almacenan en las E.D. y de destruirlos**.
8. Los archivos **.csv** con los datos que hay que importar (`pacientes.csv`, `medicos.csv` e `informes.csv`).
9. Un **programa principal** que implementa un **menú** para ejecutar diferentes algoritmos. El esqueleto de código para este menú lo podéis descargar del Aula Virtual.

A partir de la estructura anterior, se incorporará nueva funcionalidad al Hospital en forma de algoritmos que se ejecutarán a partir del menú.

Implementación de las clases base.

La elección de los atributos de cada clase y su representación (tipos de datos) es responsabilidad del programador y se deben ajustar a los datos incluidos en los archivos y a lo establecido en las prácticas de la asignatura.

Las operaciones a implementar, establecidas en las sesiones de laboratorio, serán en general para cada clase: (1) *setters/getters* para cada atributo, (2) constructores, (3) destructor, (4) operadores sobrecargados necesarios y (5) una operación para mostrar la información en pantalla. A estas operaciones básicas se podrán añadir otras si se considera necesario.

Datos.

Los datos a importar han sido pre-tratados y están a disposición del programador en el Aula Virtual de la asignatura en forma de ficheros **.csv**. Se deben utilizar los datos proporcionados en los ficheros **sin modificaciones**. Cualquier modificación supondrá que los resultados de los algoritmos no serán los esperados y, como consecuencia, podrán considerarse incorrectos. Los archivos que contienen los datos son de tipo texto y se enumeran en la tabla siguiente:

<code>pacientes.csv</code>	Fichero que contiene los pacientes del hospital. La información de cada paciente será: <i>DNI, Nombre, apellidos, género</i> (0: Hombre, 1: Mujer, 2: NoDefinido) y <i>edad</i> .
<code>medicos.csv</code>	Fichero que contiene los médicos del hospital. La información de cada médico será: <i>Nombre, apellidos y especialidad</i> .
<code>informes.csv</code>	Fichero que contiene los informes realizados sobre cada uno de los pacientes. La información, por cada línea, será: DNI paciente; texto; apellidos del médico que realizó el informe; fecha y hora del informe

Cada paciente, médico o informe aparece en una línea del fichero correspondiente, y los campos se separan por el carácter ';'.

Es labor del programador importar estos datos a la aplicación de forma correcta.

La información de los ficheros se cargará en las estructuras de datos de la aplicación al inicio de forma automática, en los constructores del Hospital (antes de mostrar el menú por pantalla). Al finalizar la aplicación, será el destructor de esta clase, el encargado de liberar la memoria asociada a todos los elementos previamente creados.

Contenedores de datos.

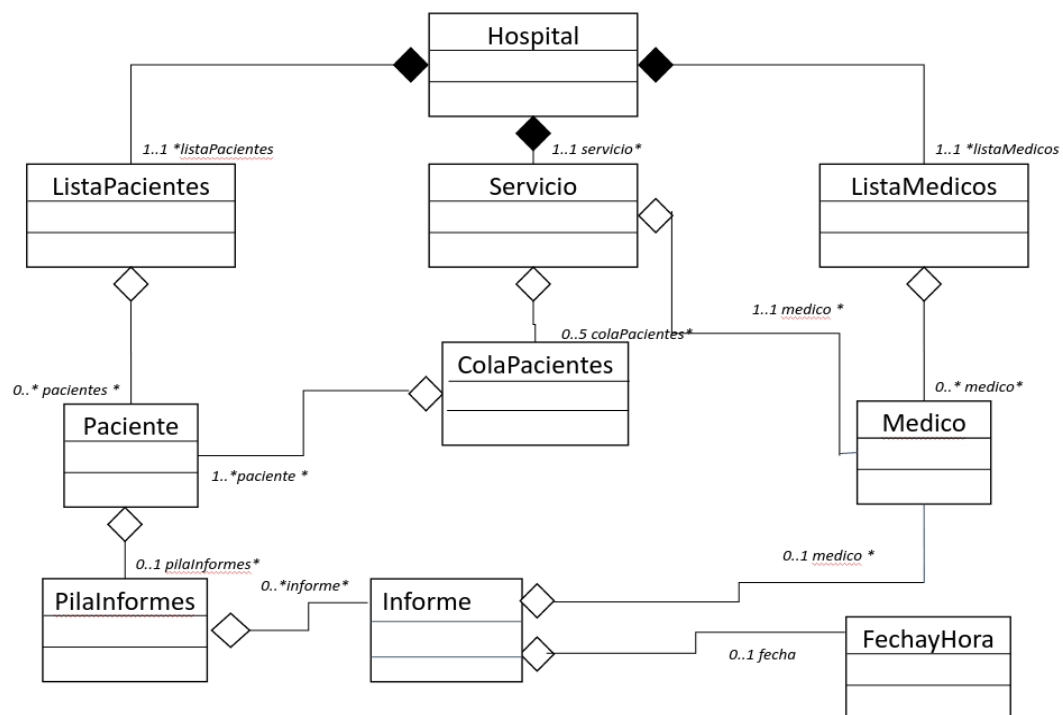
Las estructuras de datos básicas que vamos a utilizar en nuestra aplicación son la Lista con Punto de Interés, la Pila y la Cola. La implementación de dichos contenedores será proporcionada por los profesores de la asignatura y deben usarse sin modificaciones.

La E.D. ListaDPI se utilizará para almacenar los datos de los Médicos y de los Pacientes en la aplicación, la E.D. Pila se utilizará para almacenar los informes de cada uno de los pacientes y la E.D. Cola se utilizará para almacenar los pacientes en espera para ser atendidos por un Servicio. Será necesario utilizarlas correctamente siguiendo la sintaxis basada en templates de C++ y respetando lo indicado en las Precondiciones y Descripciones de cada una de sus operaciones.

Los criterios de ordenación de los datos en las estructuras de datos de la aplicación deben quedar claramente definidos. Los Pacientes se ordenarán por su DNI, los Médicos alfabéticamente por sus apellidos, los informes del más reciente al más antiguo y los Pacientes en espera de ser atendidos, en orden de llegada según su prioridad.

Relaciones entre clases.

El siguiente diagrama *UML* muestra la estructura de clases del proyecto:



3. Algoritmos a implementar.

Los algoritmos a implementar en nuestra aplicación son los siguientes:

1. **Mostrar estadísticas del hospital:** número de pacientes, número de médicos y número de pacientes en espera por cada una de las prioridades.
2. **Mostrar todos los Pacientes.**
3. **Mostrar todos los Médicos.**
4. **Mostrar todos los pacientes en espera a ser atendidos en el Servicio.**
5. **Buscar un paciente por DNI.** El método debe **devolver** (sin mostrar) el paciente que tiene el DNI solicitado. El paciente obtenido al llamar a este método (si se localiza) se mostrará fuera del método *buscar*. Mostrará también todos sus informes.
6. **Buscar un médico por apellido.** El método debe devolver (sin mostrar) el médico que tiene el apellido solicitado. El médico obtenido, como antes, se mostrará (si se localiza) fuera del método *buscar*.
7. **Asignar un médico a un servicio.** Busca en la lista de médicos un médico de la especialidad del servicio y se lo asigna al servicio para realizar las consultas.
8. **Procesar las colas de espera del Servicio, solo si tiene médico asignado.**

El proceso consiste en **ir obteniendo los pacientes de las colas según su prioridad y orden de llegada**, al paciente que se atiende se le **genera un informe que se añade a su historia médica**, se **muestra el paciente con todos sus informes y desaparece de la cola de espera**. El proceso **termina cuando no quedan pacientes en espera**.

Nota: Al generar el **informe** debéis **incluir una fecha y una hora**. Para ello, una posibilidad es obtener la fecha y hora del sistema. Os dejo una función para obtenerla y un ejemplo de como usarla. La otra posibilidad es introducir por teclado la fecha y hora del informe.

4. Análisis, pruebas y documentación.

Además del código que implementa la funcionalidad descrita en las secciones anteriores, para obtener la calificación de aprobado será obligatorio incluir:

- Juego de pruebas unitarias para cada clase implementada, incluyendo su diseño e implementación en las librerías `pruebaClase.h` y `pruebaClase.cpp`, respectivamente. Se podrá realizar una prueba conjunta de los constructores y los *setters/getters* en las clases base (*Paciente*, *Medico* y *Informe*).
- La documentación interna correspondiente a cada operación de las clases implementadas. Esta documentación incluirá *precondiciones*, *descripción* y *complejidad*. Se documentarán también los aspectos que el alumno considere oportunos sobre los detalles de su código.
- No se realizan pruebas de la clase *Hospital*.

5. Evaluación.

La rúbrica con la que se evaluará el proyecto está disponible en la actividad de entrega. Tal y como se indica en ella, los requisitos mínimos que deben cumplir los proyectos entregados para ser evaluados, son los siguientes:

- Tal y como se indica en la ficha de la asignatura, *la copia o el plagio en cualquier actividad, proyecto o prueba, ya sea en una parte o en su totalidad, supone una nota final de SUSPENSO (0) en la convocatoria y una nota de 0 en todas las calificaciones obtenidas hasta el momento*

para todas las personas implicadas, además de las actuaciones legales pertinentes.

- Aquellos proyectos que no compilen o que no funcionen correctamente en la mayoría de los casos, no se corregirán y serán calificados con una nota de 0.
- Del mismo modo, aquellos proyectos que no incluyan la documentación interna de todas las clases o pruebas significativas para todas ellas, tampoco serán corregidos y serán calificados con una nota de 0.
- Aquellos proyectos en cuyas estructuras de datos se almacenen objetos en lugar de punteros a objetos obtendrán una calificación con una nota de 0.
- Para la implementación del proyecto solo podrá utilizarse C++. No está permitido utilizar la biblioteca STL (*Standard Template Library*) en el desarrollo, ni cualquier otra biblioteca similar.

Los proyectos entregados deberán cumplir las siguientes normas:

- Deberán realizarse **de forma individual**.
- Deberán entregarse en un fichero `zip` cuyo nombre seguirá el formato `ApellidosAlumno.zip` y cuyo contenido incluirá los archivos fuente (`.h` y `.cpp`) del código y pruebas desarrolladas, con su documentación interna.

Aquellos proyectos que cumplan los requisitos y normas anteriores, se evaluarán teniendo en cuenta los siguientes criterios:

- Uso de **punteros** en estructuras de datos, evitando así los problemas derivados de mover datos en memoria (*eficiencia*) y mantener distintas copias (*consistencia*).
- Diseño correcto de interfaces, incluidas la documentación (*precondiciones* y *descripción*) y evaluación de la *complejidad* de las operaciones.
- Organización correcta de los datos en memoria de forma que la implementación de los algoritmos cumpla con los requisitos de eficiencia y uso de memoria.
- Corrección en el resultado de la ejecución de los algoritmos.
- Uso correcto y eficiente de parámetros en la invocación de operaciones.
- Uso de constructores y destructores para las clases de forma coherente y adecuada, incluido el constructor por copia.
- Siempre que sea posible, no se duplicarán objetos en la aplicación, y se evitarán copias y duplicados de los mismos.

La fecha límite de entrega del proyecto es el **2 de abril 2023 a las 23:55 h**, a través de la correspondiente tarea abierta en el Aula Virtual de la asignatura. No se tendrán en cuenta entregas realizadas por cualquier otro medio que no sea el establecido, ni con posterioridad a esa fecha.